UNIVERSITÉ DE MONS

FACULTÉ DES SCIENCES

A Refactoring Tool for GitHub Actions Workflows

Abdellah BOUMAIZA

Master en Sciences Informatiques

abdellah.boumaiza@student.umons.ac.be

Academic year 2026–2027

Master's thesis carried out in the **Software Engineering Lab**, as part of the **Master en Sciences Informatiques**.

Supervisor: Prof. Tom MENS
Co-supervisor: Aref TALEBZADEH BARDSIRI

Faculté des Sciences

UMONS Université de Mons

The author, Abdellah BOUMAIZA, declares having complied with the ethical rules in force, including the University's charter on the use of Artificial Intelligence.

Use of AI tools

Generative AI tools (Claude, Anthropic) were used to help structure the document, draft the LaTeX skeleton and continuous-integration workflow of this repository, propose a first version of the refactoring exemplars, and rephrase parts of the text. All content was reviewed, checked against the cited sources and is the responsibility of the author.

Contents

Introduction **4**

1 Reuse mechanisms in GitHub Actions **5**

2 A catalogue of reuse refactorings **6**
2.1 R1 – Extract Composite Action 6
2.2 R2 – Extract Reusable Workflow 8
2.3 R3 – Introduce YAML Anchors 10
2.4 R4 – Extract Workflow Template 11
2.5 R5 – Split Workflow 12
2.6 R6 – Introduce Matrix 13

3 Conclusion **15**

Bibliography **16**

Introduction

GitHub Actions has become the dominant continuous-integration and automation service for repositories hosted on GitHub [1, 6]. Workflows are YAML files that tend to grow and to be duplicated within and across repositories. As for source code, this duplication hampers maintenance and can be addressed by *refactoring*, i.e., behaviour-preserving transformations that improve the structure of an artefact [2, 7].

This thesis focuses on refactorings that improve *reuse* in workflows. Its goals are (i) to catalogue the reuse mechanisms offered by GitHub Actions and the refactorings they enable (Chapters 1 and 2), and (ii) to build a tool that detects refactoring opportunities and applies them automatically.

Chapter 1

Reuse mechanisms in GitHub Actions

GitHub Actions offers several mechanisms to avoid duplicating workflow configuration [5]. They differ in the *granularity* of what is reused (a YAML node, a sequence of steps, a job, a whole workflow), in their *scope* (one file, one repository, an organisation, the whole platform) and in the moment at which reuse happens (parse time, run time, or creation time). Table 1.1 summarises them.

Mechanism	**Reused unit**	**Scope**	**When**
YAML anchors/aliases [3, 8]	any YAML node	one file	parse time
Composite action [4]	sequence of steps	repo / public	run time
JavaScript / Docker action	one step (custom code)	repo / public	run time
Reusable workflow	one or more jobs	repo / org / public	run time
Workflow template [3]	whole workflow file	organisation	creation time
Matrix strategy	one job, parametrised	one job	run time
Chained workflows (`workflow_run`)	a set of jobs	one repository	run time

Table 1.1: Reuse mechanisms available in GitHub Actions.

Chapter 2

A catalogue of reuse refactorings

Each refactoring is illustrated by a pair of artificial but realistic workflows (*before* / *after*) stored in the `examples/` folder of the repository. These exemplars form a small ground truth: they document the catalogue and will serve as a test suite for the refactoring tool. All of them are validated by `actionlint`, and, where the refactoring is a pure syntactic rewriting, an automatic equivalence check is provided (`examples/check.py`).

Id	Refactoring	Strictly behaviour-preserving?
R1	Extract Composite Action	yes
R2	Extract Reusable Workflow	yes (modulo UI/log structure)
R3	Introduce YAML Anchors	yes (identical after alias expansion)
R4	Extract Workflow Template	yes for the instantiated file; no propagation
R5	Split Workflow (`workflow_run`)	no: default-branch version, security context
R6	Introduce Matrix	yes with `fail-fast: false`; job ids change

Table 2.1: Refactorings of the catalogue.

2.1 R1 – Extract Composite Action

Smell. The same sequence of steps is repeated in several jobs. **Mechanism.** Composite action [4].

```yaml
name: CI

on:
  push:
    branches: [main]
  pull_request:

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm
      - name: Install dependencies
        run: npm ci --no-audit --no-fund
      - name: Lint
        run: npm run lint

  test:
```

```
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm
      - name: Install dependencies
        run: npm ci --no-audit --no-fund
      - name: Unit tests
        run: npm test -- --coverage
      - uses: actions/upload-artifact@v4
        with:
          name: coverage
          path: coverage/

  build:
    runs-on: ubuntu-latest
    needs: [lint, test]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm
      - name: Install dependencies
        run: npm ci --no-audit --no-fund
      - name: Build
        run: npm run build
      - uses: actions/upload-artifact@v4
        with:
          name: dist
          path: dist/
```

Listing 2.1: R1, before: three jobs repeat the Node setup.

```
name: Setup Node project
# Note: a *local* composite action cannot perform the checkout itself, because
# the action's own files only exist on the runner after actions/checkout ran.
description: Install Node.js (with npm cache) and install npm dependencies.

inputs:
  node-version:
    description: Node.js version to install
    required: false
    default: "20"

runs:
  using: composite
  steps:
    - uses: actions/setup-node@v4
      with:
        node-version: ${{ inputs.node-version }}
        cache: npm
    - name: Install dependencies
      shell: bash
      run: npm ci --no-audit --no-fund
```

Listing 2.2: R1, after: extracted composite action.

```
name: CI

on:
  push:
    branches: [main]
  pull_request:

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
```

```
      - uses: ./.github/actions/setup-node-project
      - name: Lint
        run: npm run lint

  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: ./.github/actions/setup-node-project
      - name: Unit tests
        run: npm test -- --coverage
      - uses: actions/upload-artifact@v4
        with:
          name: coverage
          path: coverage/

  build:
    runs-on: ubuntu-latest
    needs: [lint, test]
    steps:
      - uses: actions/checkout@v4
      - uses: ./.github/actions/setup-node-project
      - name: Build
        run: npm run build
      - uses: actions/upload-artifact@v4
        with:
          name: dist
          path: dist/
```

Listing 2.3: R1, after: the workflow calls the local action.

Pitfalls. `run` steps must declare a `shell`; a local action is only available after `actions/checkout`; job-level keys (`runs-on`, `services`, `permissions`) cannot be extracted.

2.2 R2 – Extract Reusable Workflow

Smell. The same *job* is duplicated in several workflows. **Mechanism.** Reusable workflow (`on: workflow_call`) [5].

```
name: Deploy to staging

on:
  push:
    branches: [develop]

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: staging
    permissions:
      contents: read
      id-token: write
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-buildx-action@v3
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - name: Build and push image
        uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: ghcr.io/${{ github.repository }}:staging-${{ github.sha }}
      - name: Deploy
        run: |
```

```
          ./scripts/deploy.sh \
            --env staging \
            --image ghcr.io/${{ github.repository }}:staging-${{ github.sha }} \
            --replicas 1
        env:
          DEPLOY_TOKEN: ${{ secrets.STAGING_DEPLOY_TOKEN }}
      - name: Smoke test
        run: curl --fail --retry 5 --retry-delay 10 https://staging.example.org/health
```

Listing 2.4: R2, before: staging deployment (production is identical up to 5 values).

```
name: Deploy (reusable)

on:
  workflow_call:
    inputs:
      environment:
        description: Target GitHub environment (staging | production)
        type: string
        required: true
      image-tag:
        type: string
        required: true
      replicas:
        type: number
        default: 1
      health-url:
        type: string
        required: true
    secrets:
      deploy-token:
        required: true

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: ${{ inputs.environment }}
    permissions:
      contents: read
      id-token: write
    env:
      IMAGE: ghcr.io/${{ github.repository }}:${{ inputs.image-tag }}
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-buildx-action@v3
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - name: Build and push image
        uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: ${{ env.IMAGE }}
      - name: Deploy
        run: |
          ./scripts/deploy.sh \
            --env ${{ inputs.environment }} \
            --image "$IMAGE" \
            --replicas ${{ inputs.replicas }}
        env:
          DEPLOY_TOKEN: ${{ secrets.deploy-token }}
      - name: Smoke test
        run: curl --fail --retry 5 --retry-delay 10 ${{ inputs.health-url }}
```

Listing 2.5: R2, after: the reusable workflow.

```
name: Deploy to staging

on:
```

```yaml
  push:
    branches: [develop]

jobs:
  deploy:
    permissions:
      contents: read
      id-token: write
    uses: ./.github/workflows/_deploy.yml
    with:
      environment: staging
      image-tag: staging-${{ github.sha }}
      replicas: 1
      health-url: https://staging.example.org/health
    secrets:
      deploy-token: ${{ secrets.STAGING_DEPLOY_TOKEN }}
```

Listing 2.6: R2, after: a caller.

2.3 R3 – Introduce YAML Anchors

Smell. Repeated YAML nodes *within one file*. **Mechanism.** YAML anchors and aliases [8], supported by GitHub Actions since September 2025 [3].

```yaml
name: Python CI

on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    env: &ci-env
      PYTHONUNBUFFERED: "1"
      PIP_DISABLE_PIP_VERSION_CHECK: "1"
      DATABASE_URL: postgresql://ci:ci@localhost:5432/app
    services: &ci-services
      postgres:
        image: postgres:16
        env:
          POSTGRES_USER: ci
          POSTGRES_PASSWORD: ci
          POSTGRES_DB: app
        ports: ["5432:5432"]
        options: >-
          --health-cmd "pg_isready -U ci"
          --health-interval 10s
          --health-retries 5
    steps:
      - &checkout
        uses: actions/checkout@v4
      - &setup-python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip
      - &install-deps
        run: pip install -r requirements.txt -r requirements-dev.txt
      - run: pytest tests/unit

  integration:
    runs-on: ubuntu-latest
    env: *ci-env
    services: *ci-services
    steps:
      - *checkout
```

```
      - *setup-python
      - *install-deps
      - run: pytest tests/integration --maxfail=1

  migrations:
    runs-on: ubuntu-latest
    env: *ci-env
    services: *ci-services
    steps:
      - *checkout
      - *setup-python
      - *install-deps
      - run: alembic upgrade head && alembic check
```

Listing 2.7: R3, after (the *before* version, with the three blocks inlined three times, is in the repository).

Remark. Anchors are expanded at parse time, so behaviour preservation can be checked mechanically by comparing the parsed documents. Anchors are file-local and not parametrisable.

2.4 R4 – Extract Workflow Template

Smell. The same workflow file is copy-pasted across repositories of an organisation. **Mechanism.** Organisation workflow template, possibly in a non-public `.github` repository since 2025 [3].

```
name: Java CI

on:
  push:
    branches: [ $default-branch ]
  pull_request:

permissions:
  contents: read

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: "21"
          cache: maven
      - name: Build and test
        run: mvn -B -ntp verify
      - name: Upload test reports
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: surefire-reports
          path: target/surefire-reports/
```

Listing 2.8: R4, the template (`$default-branch` is a placeholder).

```
{
  "name": "Java CI (Maven, organization standard)",
  "description": "Build and test a Maven project with Temurin 21, and keep the test reports.",
  "iconName": "java-maven-ci",
  "categories": ["Java", "Maven"],
  "filePatterns": ["pom\\.xml$"]
}
```

Listing 2.9: R4, template metadata.

Remark. Unlike R1–R3, a template is *copied* at creation time: duplication is standardised rather than removed, and later changes are not propagated. Combining R4 with R2 (a template that only calls a reusable workflow) gives both.

2.5 R5 – Split Workflow

Smell. A long sequential workflow mixes several concerns. **Mechanism.** Two regular workflows chained with the `workflow_run` event.

```yaml
name: CI and docs

on:
  push:
    branches: [main]
  pull_request:

permissions:
  contents: read

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install build
      - run: python -m build
      - uses: actions/upload-artifact@v4
        with:
          name: dist
          path: dist/

  test:
    runs-on: ubuntu-latest
    needs: build
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - uses: actions/download-artifact@v4
        with:
          name: dist
          path: dist/
      - run: pip install dist/*.whl pytest
      - run: pytest

  build-docs:
    runs-on: ubuntu-latest
    needs: test
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -r docs/requirements.txt
      - run: sphinx-build -W -b html docs/ site/
      - uses: actions/upload-pages-artifact@v3
        with:
          path: site/

  deploy-docs:
    runs-on: ubuntu-latest
    needs: build-docs
    permissions:
      pages: write
```

```
      id-token: write
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    steps:
      - id: deployment
        uses: actions/deploy-pages@v4
```

Listing 2.10: R5, before.

```
name: Docs

on:
  workflow_run:
    workflows: [CI]          # must match the ‘name:‘ of ci.yml
    types: [completed]
    branches: [main]

permissions:
  contents: read

jobs:
  build-docs:
    runs-on: ubuntu-latest
    # Reproduces the old ‘needs: test‘ + ‘if:‘ condition:
    # only after a *successful* CI run triggered by a *push* on main.
    if: >-
      github.event.workflow_run.conclusion == 'success' &&
      github.event.workflow_run.event == 'push'
    steps:
      - uses: actions/checkout@v4
        with:
          # check out the commit CI ran on, not the latest commit of main
          ref: ${{ github.event.workflow_run.head_sha }}
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -r docs/requirements.txt
      - run: sphinx-build -W -b html docs/ site/
      - uses: actions/upload-pages-artifact@v3
        with:
          path: site/

  deploy-docs:
    runs-on: ubuntu-latest
    needs: build-docs
    permissions:
      pages: write
      id-token: write
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    steps:
      - id: deployment
        uses: actions/deploy-pages@v4
```

Listing 2.11: R5, after: the second workflow.

Remark. This refactoring is *not* strictly behaviour-preserving: the triggered workflow always uses the default-branch version of its file and runs in a privileged context, even for runs triggered by pull requests from forks.

2.6 R6 – Introduce Matrix

Smell. Jobs that differ only by a few values. **Mechanism.** `strategy.matrix`.

```
name: Tests
```

```yaml

on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    name: test (${{ matrix.os }}, node ${{ matrix.node }})
    runs-on: ${{ matrix.os }}
    strategy:
      fail-fast: false      # independent jobs in ‘before/‘ did not cancel each other
      matrix:
        os: [ubuntu-latest, windows-latest, macos-latest]
        node: [18, 20, 22]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node }}
          cache: npm
      - run: npm ci
      - run: npm test
```

Listing 2.12: R6, after (the *before* version contains nine near-identical jobs).

Chapter 3

Conclusion

Bibliography

[1] Alexandre Decan, Tom Mens, Pooya Rostami Mazrae, and Mehdi Golzadeh. On the use of GitHub Actions in software development repositories. In *Proceedings of the IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 235–245. IEEE, 2022.

[2] Martin Fowler. *Refactoring: Improving the Design of Existing Code*. Addison-Wesley, 2nd edition, 2018.

[3] GitHub. Actions YAML anchors and non-public workflow templates. GitHub Changelog, `https://github.blog/changelog/2025-09-18-actions-yaml-anchors-and-non-public-workflow-templates/`, September 2025. Accessed 15 October 2026.

[4] GitHub. Creating a composite action. GitHub Docs, `https://docs.github.com/actions/sharing-automations/creating-actions/creating-a-composite-action`, 2026. Accessed 15 October 2026.

[5] GitHub. Reusing workflow configurations. GitHub Docs, `https://docs.github.com/actions/reference/workflows-and-actions/reusing-workflow-configurations`, 2026. Accessed 15 October 2026.

[6] Timothy Kinsman, Mairieli Wessel, Marco A. Gerosa, and Christoph Treude. How do software developers use GitHub Actions to automate their workflows? In *Proceedings of the 18th International Conference on Mining Software Repositories (MSR)*, pages 420–431. IEEE, 2021.

[7] Tom Mens and Tom Tourwé. A survey of software refactoring. *IEEE Transactions on Software Engineering*, 30(2):126–139, 2004.

[8] YAML Language Development Team. YAML ain't markup language (YAML) version 1.2.2. `https://yaml.org/spec/1.2.2/`, 2021.